Static analysis and software verification

Lecture 8 - Introduction to abstract interpretation (Abstract semantics)

Vincenzo Arceri - University of Parma - vincenzo.arceri@unipr.it

Concrete semantics

Concrete semantics

- In lecture 2 we already introduce the concrete and collecting semantics for IMP language

Concrete semantics

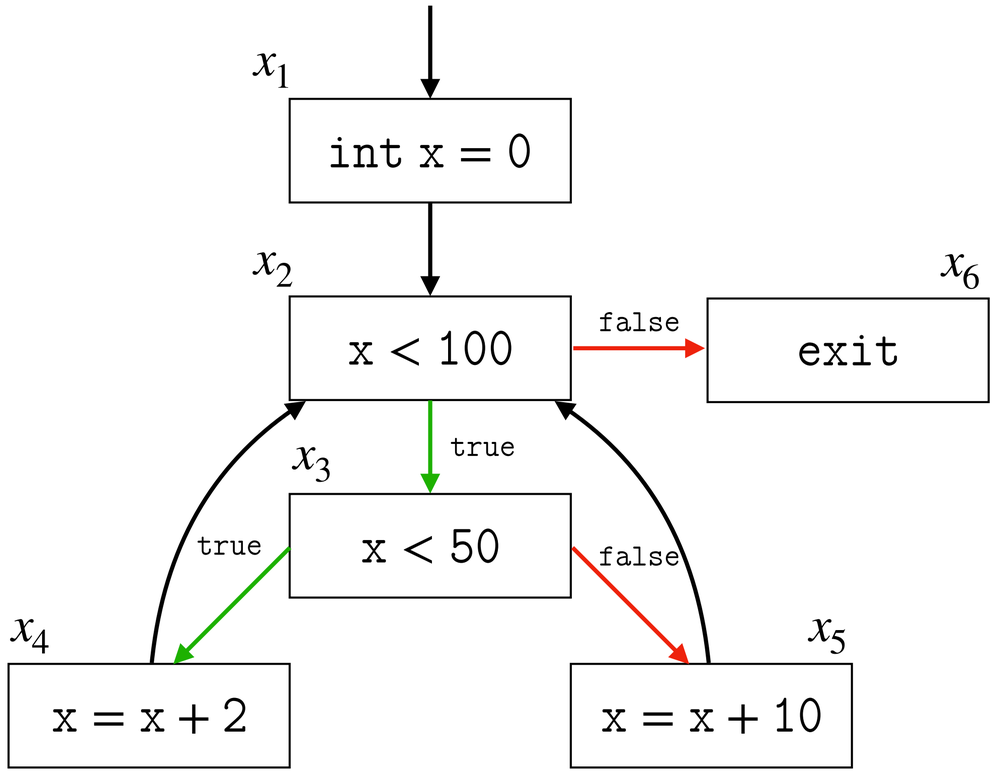
- In lecture 2 we already introduce the concrete and collecting semantics for IMP language
- We talking about abstract semantics, we want to encode the collecting semantics over CFGs (rather than over the source code program)

Concrete semantics

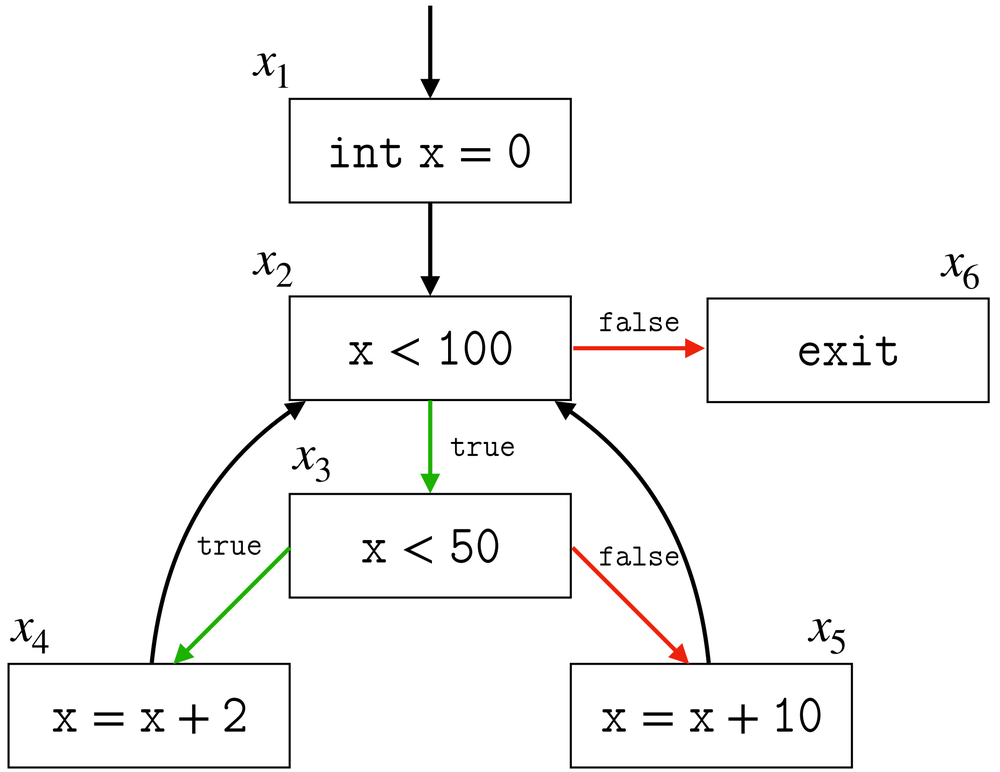
- In lecture 2 we already introduce the concrete and collecting semantics for IMP language
- We talking about abstract semantics, we want to encode the collecting semantics over CFGs (rather than over the source code program)
- `while` and `if` semantics is directly encoded in the CFG (and fixpoint computation over the CFG)

```
1 x = 0;
2 while (x < 100)
3    if (x < 50)
4       x = x + 2;
5    else
6       x = x + 10;
```

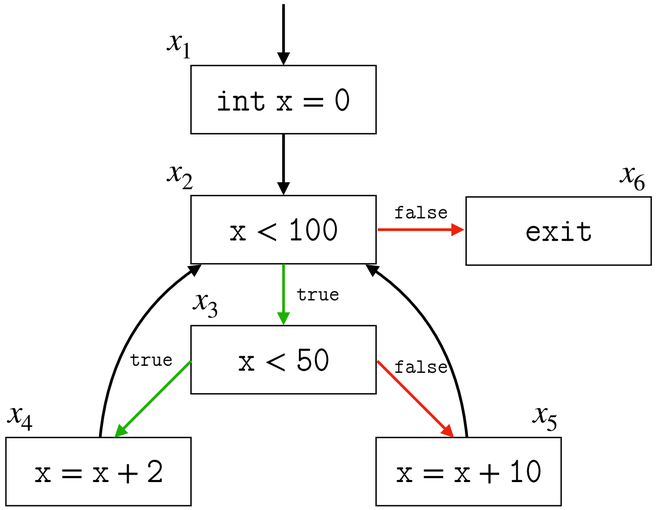
```
1 x = 0;
2 while (x < 100)
3    if (x < 50)
4       x = x + 2;
5    else
6       x = x + 10;
```

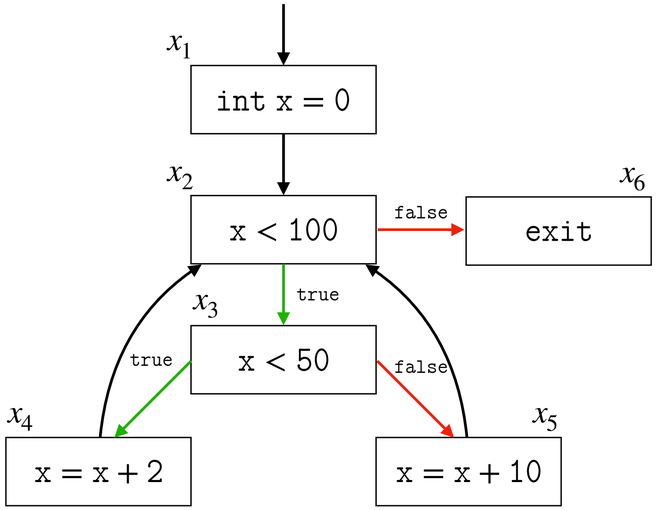


```
1 x = 0;
2 while (x < 100)
3    if (x < 50)
4       x = x + 2;
5    else
6       x = x + 10;
```

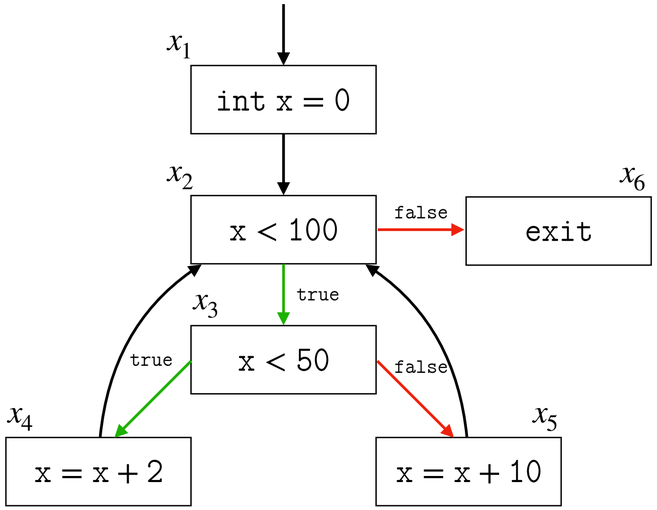


- The goal is to infer the entry/exit state for each program point x_i (point of interest of the analysis)

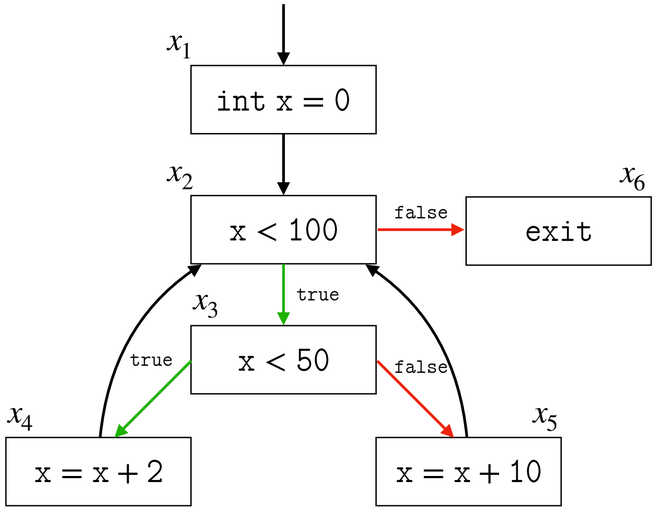

x_1
int x = 0
x_2
x < 100
false
x_6
exit
true
x_3
x < 50
true
false
x_4
x = x + 2
x_5
x = x + 10



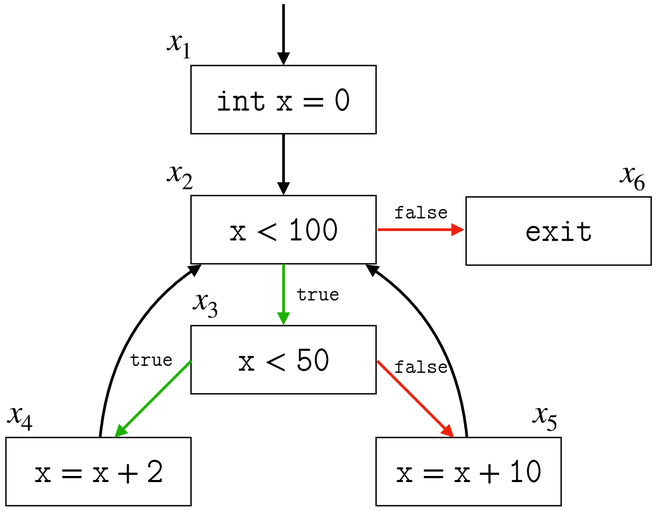
- Entry state of a block: upper bound of the exit states of blocks pointing to it



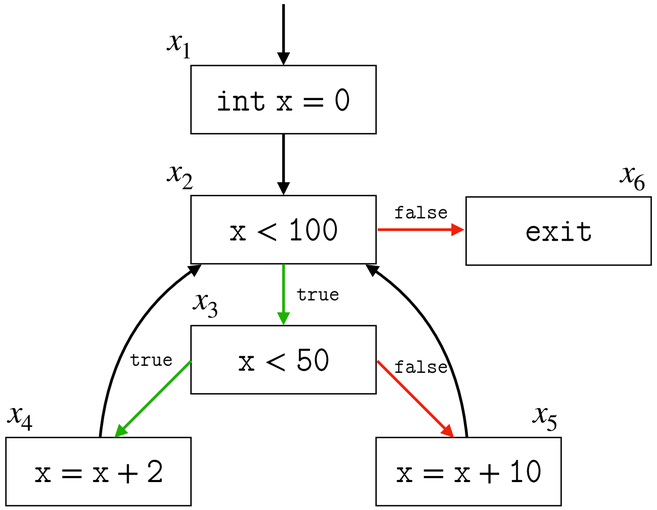
- Entry state of a block: upper bound of the exit states of blocks pointing to it
  - First block: the given entry state(s)



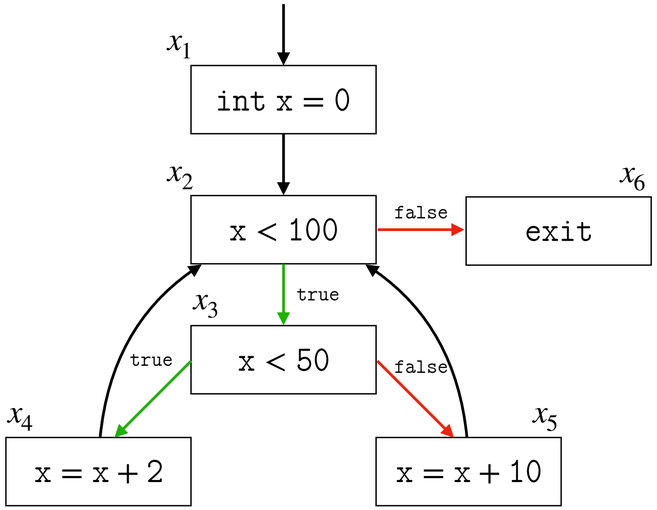
- Entry state of a block: upper bound of the exit states of blocks pointing to it
  - First block: the given entry state(s)
- Exit state: the application of the (collecting) semantics of the statement contained in the block applied to the entry state



- Entry state of a block: upper bound of the exit states of blocks pointing to it
  - First block: the given entry state(s)
- Exit state: the application of the (collecting) semantics of the statement contained in the block applied to the entry state
- `while`, `if` and statement concatenation semantics encoded in the CFG



- Entry state of a block: upper bound of the exit states of blocks pointing to it
  - First block: the given entry state(s)
- Exit state: the application of the (collecting) semantics of the statement contained in the block applied to the entry state
- `while`, `if` and statement concatenation semantics encoded in the CFG
  - Thus, we need to express the semantics just over assignments and boolean expressions



$x_1 =$

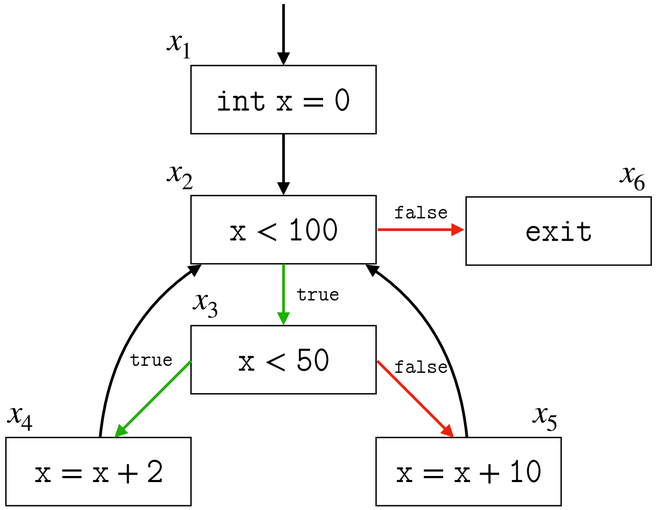
$x_2 =$

$x_3 =$

$x_4 =$

$x_5 =$

$x_6 =$



$$x_1 = I$$

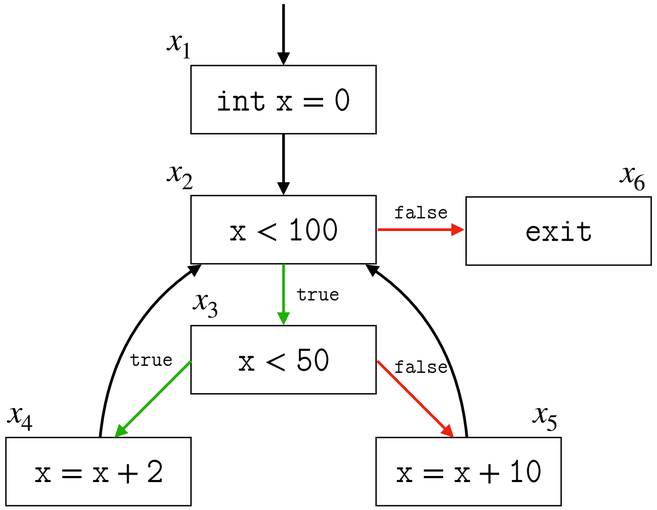
$$x_2 =$$

$$x_3 =$$

$$x_4 =$$

$$x_5 =$$

$$x_6 =$$



$$x_1 = I$$

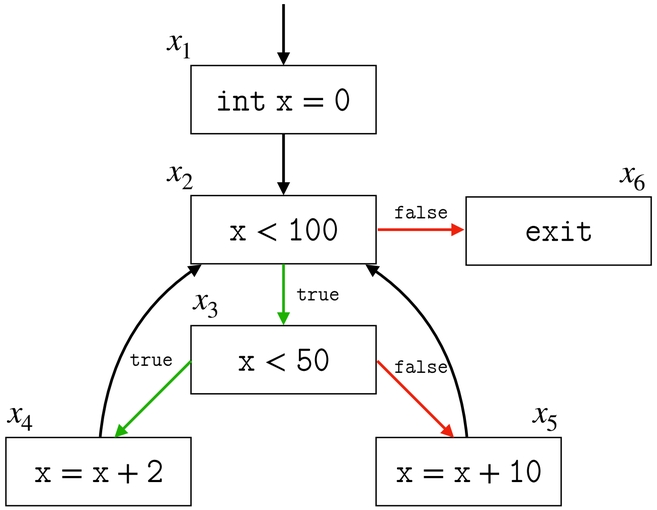
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$

$$x_3 =$$

$$x_4 =$$

$$x_5 =$$

$$x_6 =$$



$$x_1 = I$$

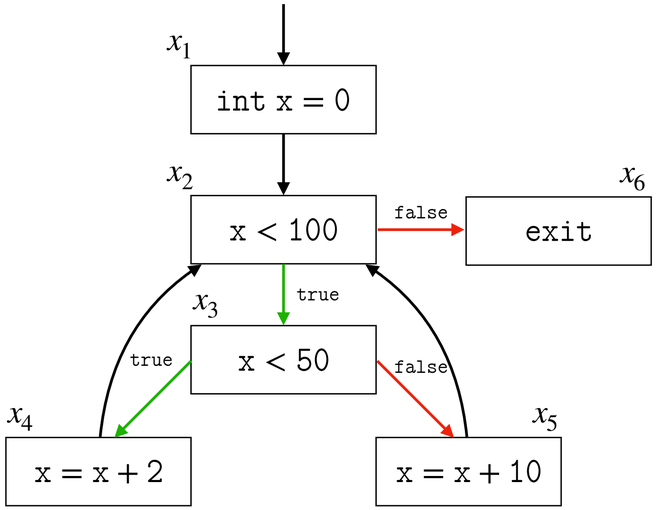
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$

$$x_3 = [[\mathrm{x} < 100]]x_2$$

$$x_4 =$$

$$x_5 =$$

$$x_6 =$$



$$x_1 = I$$

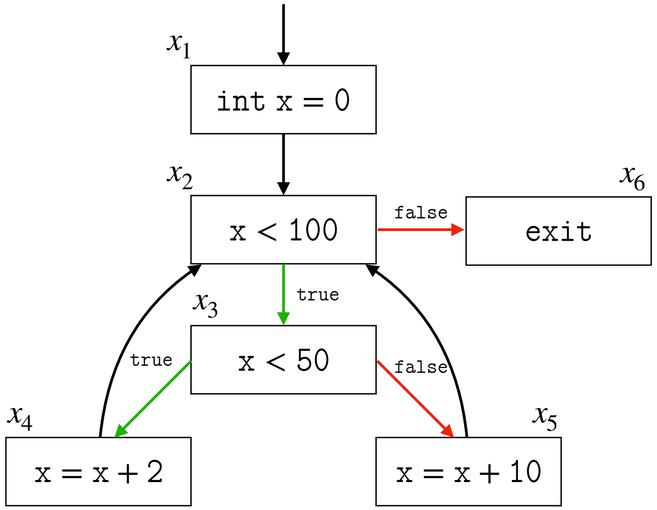
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$

$$x_3 = [[\mathrm{x} < 100]]x_2$$

$$x_4 = [[\mathrm{x} < 50]]x_3$$

$$x_5 =$$

$$x_6 =$$



$$x_1 = I$$

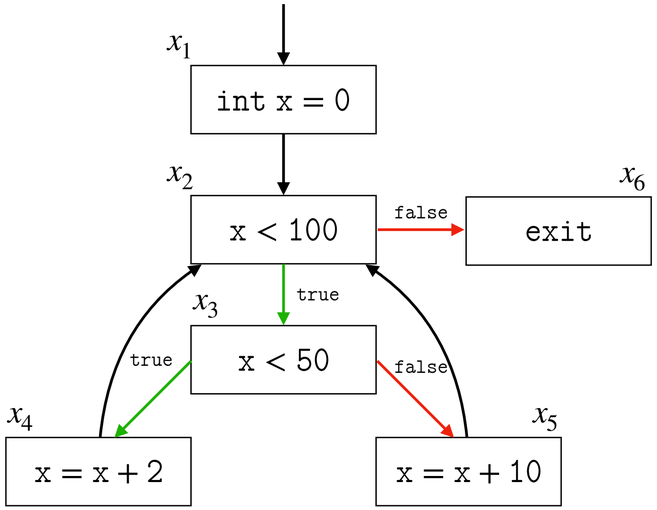
$$x_2 = [[x = 0]]x_1 \cup [[x = x + 2]]x_4 \cup [[x = x + 10]]x_5$$

$$x_3 = [[x < 100]]x_2$$

$$x_4 = [[x < 50]]x_3$$

$$x_5 = [[x >= 50]]x_3$$

$$x_6 =$$



$$x_1 = I$$

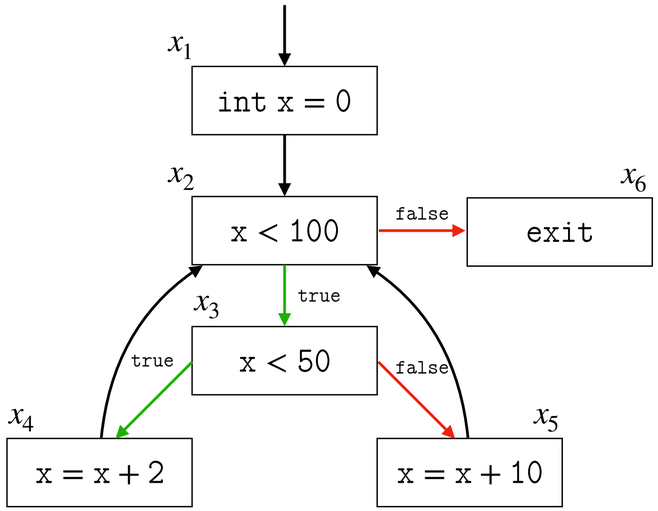
$$x_2 = [[x = 0]]x_1 \cup [[x = x + 2]]x_4 \cup [[x = x + 10]]x_5$$

$$x_3 = [[x < 100]]x_2$$

$$x_4 = [[x < 50]]x_3$$

$$x_5 = [[x >= 50]]x_3$$

$$x_6 = [[x >= 100]]x_2$$



$$x_1 = I$$

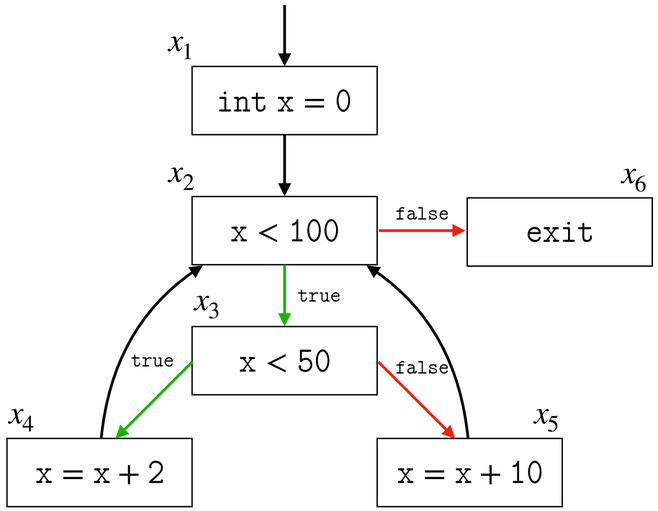
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$

$$x_3 = [[\mathrm{x} < 100]]x_2$$

$$x_4 = [[\mathrm{x} < 50]]x_3$$

$$x_5 = [[x >= 50]]x_3$$

$$x_6 = [[\mathrm{x} > 100]]x_2$$



$$x_1 = I$$

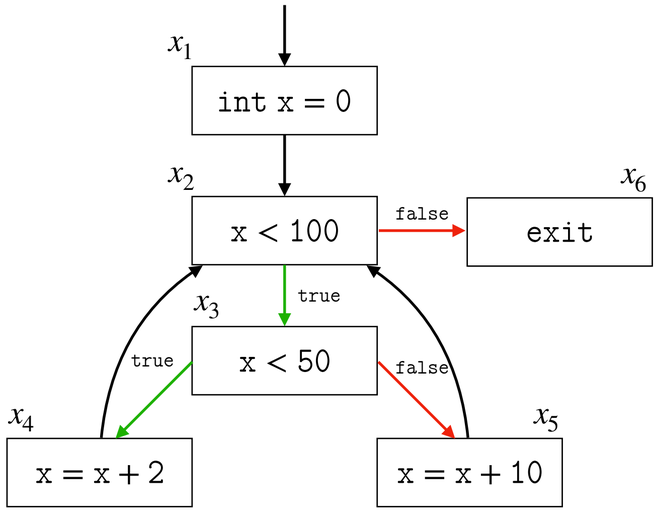
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$

$$x_3 = [[\mathrm{x} < 100]]x_2$$

$$x_4 = [[\mathrm{x} < 50]]x_3$$

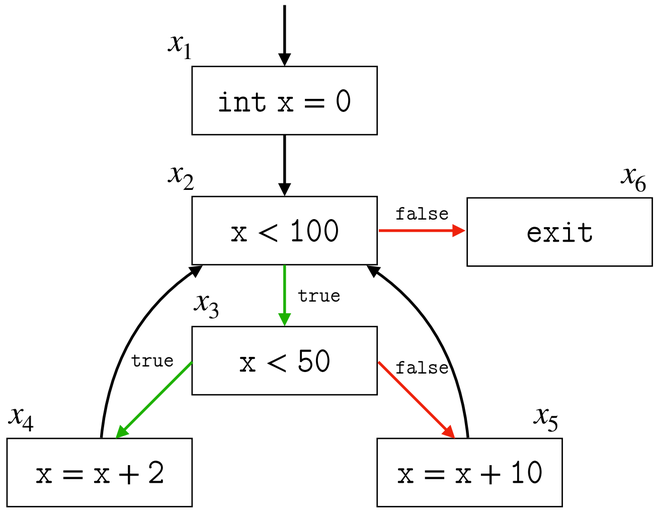
$$x_5 = [[x >= 50]]x_3$$

$$x_6 = [[\mathrm{x} > 100]]x_2$$



$$x_1 = I$$
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$
$$x_3 = [[\mathrm{x} < 100]]x_2$$
$$x_4 = [[\mathrm{x} < 50]]x_3$$
$$x_5 = [[x >= 50]]x_3$$
$$x_6 = [[\mathrm{x} > 100]]x_2$$

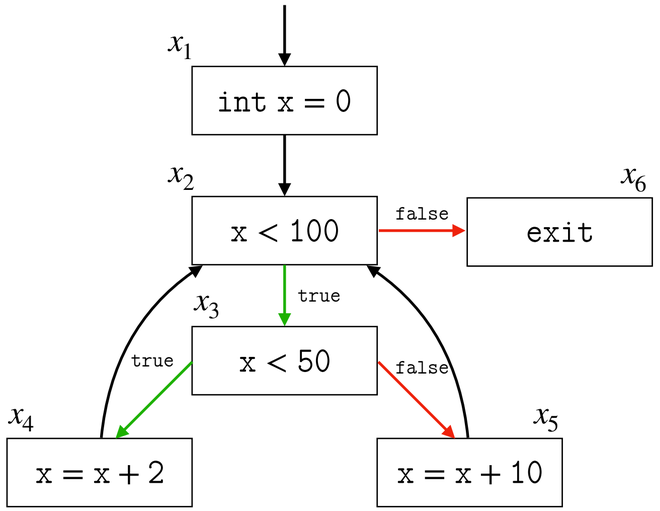
- Given m nodes of a CFG, what we would like to to is to solve the following system of equations



$$x_1 = I$$
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$
$$x_3 = [[\mathrm{x} < 100]]x_2$$
$$x_4 = [[\mathrm{x} < 50]]x_3$$
$$x_5 = [[x >= 50]]x_3$$
$$x_6 = [[\mathrm{x} > 100]]x_2$$

- Given m nodes of a CFG, what we would like to to is to solve the following system of equations

$$F = \{x_i = f_i(x_1, \ldots, x_m) \mid i = 1,2,\ldots, m\}$$

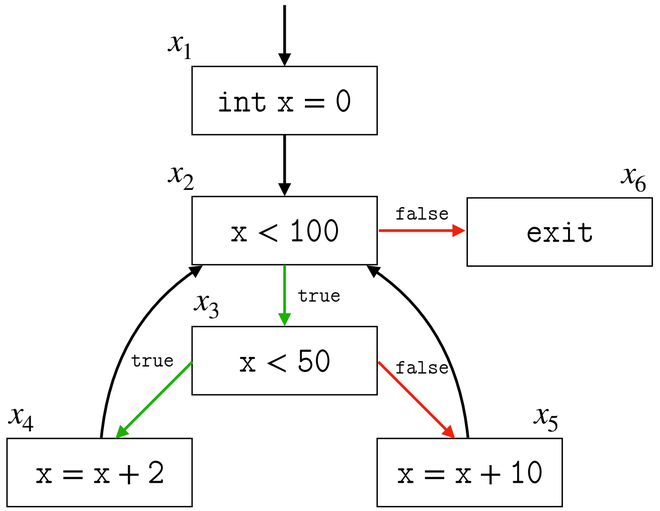


$$x_1 = I$$
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$
$$x_3 = [[\mathrm{x} < 100]]x_2$$
$$x_4 = [[\mathrm{x} < 50]]x_3$$
$$x_5 = [[x >= 50]]x_3$$
$$x_6 = [[\mathrm{x} > 100]]x_2$$

- Given m nodes of a CFG, what we would like to to is to solve the following system of equations

$$F = \{x_i = f_i(x_1, \ldots, x_m) \mid i = 1,2,\ldots, m\}$$

- computing the least solution of the equation set F as the limit of a Kleene iteration



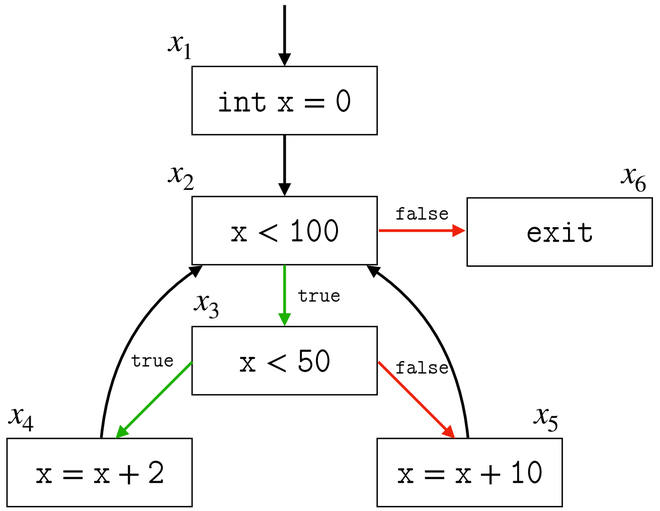
$$x_1 = I$$
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$
$$x_3 = [[\mathrm{x} < 100]]x_2$$
$$x_4 = [[\mathrm{x} < 50]]x_3$$
$$x_5 = [[x >= 50]]x_3$$
$$x_6 = [[\mathrm{x} > 100]]x_2$$

- Given m nodes of a CFG, what we would like to to is to solve the following system of equations

$$F = \{x_i = f_i(x_1, \ldots, x_m) \mid i = 1,2,\ldots, m\}$$

- computing the least solution of the equation set F as the limit of a Kleene iteration on

$$\langle \wp(\mathbb{Z}), \subseteq, \cap, \cup, \mathbb{Z}, \emptyset \rangle$$



$$x_1 = I$$
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$
$$x_3 = [[\mathrm{x} < 100]]x_2$$
$$x_4 = [[\mathrm{x} < 50]]x_3$$
$$x_5 = [[x >= 50]]x_3$$
$$x_6 = [[\mathrm{x} > 100]]x_2$$

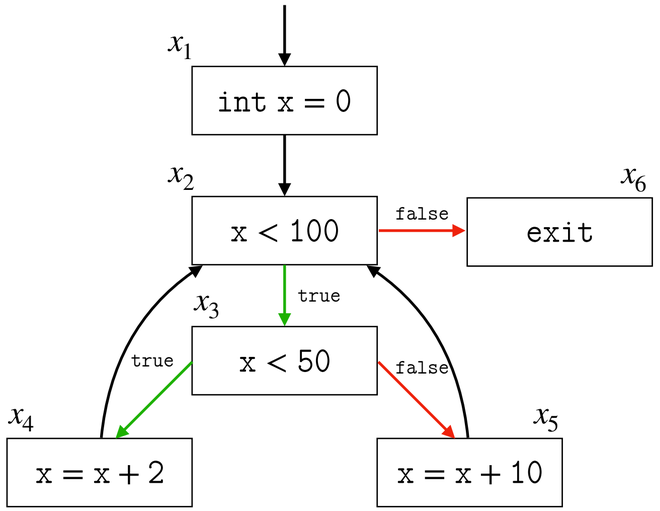
- Given m nodes of a CFG, what we would like to to is to solve the following system of equations

$$F = \{x_i = f_i(x_1, \ldots, x_m) \mid i = 1,2,\ldots, m\}$$

- computing the least solution of the equation set F as the limit of a Kleene iteration on

$$\langle \wp(\mathbb{Z}), \subseteq , \cap , \cup , \mathbb{Z}, \emptyset \rangle$$

$$x^{\mathrm{lfp(F)}} = (x_1^{\mathrm{lfp}(F)}, \ldots, x_m^{\mathrm{lfp}(F)})$$



$$x_1 = I$$
$$x_2 = [[\mathrm{x} = 0]]x_1 \cup [[\mathrm{x} = \mathrm{x} + 2]]x_4 \cup [[\mathrm{x} = \mathrm{x} + 10]]x_5$$
$$x_3 = [[\mathrm{x} < 100]]x_2$$
$$x_4 = [[\mathrm{x} < 50]]x_3$$
$$x_5 = [[x >= 50]]x_3$$
$$x_6 = [[\mathrm{x} > 100]]x_2$$

- Given m nodes of a CFG, what we would like to to is to solve the following system of equations

$$F = \{x_i = f_i(x_1, \ldots, x_m) \mid i = 1,2,\ldots, m\}$$

- computing the least solution of the equation set F as the limit of a Kleene iteration on

$$\langle \wp(\mathbb{Z}), \subseteq , \cap , \cup , \mathbb{Z}, \emptyset \rangle$$

Undecidable!

$$x^{\mathrm{lfp(F)}} = (x_1^{\mathrm{lfp}(F)}, \ldots, x_m^{\mathrm{lfp}(F)})$$

Kleene iterations on CFG

Example

Fixpoint computation

Fixpoint computation

- In general, the least fixpoint solution over the concrete domain C is not finitely computable. Hence the idea is to compute an *abstract* fixpoint, over an abstract domain A, that *correctly* approximates the concrete one

Fixpoint computation

- In general, the least fixpoint solution over the concrete domain C is not finitely computable. Hence the idea is to compute an *abstract* fixpoint, over an abstract domain A, that *correctly* approximates the concrete one
- What we would like to do is to

Fixpoint computation

- In general, the least fixpoint solution over the concrete domain C is not finitely computable. Hence the idea is to compute an *abstract* fixpoint, over an abstract domain A, that *correctly* approximates the concrete one
- What we would like to do is to
  - Compute $\alpha(\mathrm{lfp}(F))$

Fixpoint computation

- In general, the least fixpoint solution over the concrete domain C is not finitely computable. Hence the idea is to compute an *abstract* fixpoint, over an abstract domain A, that *correctly* approximates the concrete one
- What we would like to do is to
  - Compute $\alpha(\text{lfp}(F))$
  - without computing $\text{lfp}(F)$

Fixpoint computation

- In general, the least fixpoint solution over the concrete domain C is not finitely computable. Hence the idea is to compute an *abstract* fixpoint, over an abstract domain A, that *correctly* approximates the concrete one
- What we would like to do is to
  - Compute $\alpha(\text{lfp}(F))$
  - without computing $\text{lfp}(F)$
- We will later introduce two important theorems allowing us to do it

Fixpoint computation

- In general, the least fixpoint solution over the concrete domain C is not finitely computable. Hence the idea is to compute an *abstract* fixpoint, over an abstract domain A, that *correctly* approximates the concrete one
- What we would like to do is to
  - Compute $\alpha(\text{lfp}(F))$
  - without computing $\text{lfp}(F)$
- We will later introduce two important theorems allowing us to do it
- … but the first step is to move the computation from C to A

Abstract semantics

- Since we do static analysis on CFGs, it is enough to reason about assignments and boolean expressions, being the only expressions/statements appearing in the CFG

$$
\begin{aligned}
e &::= \quad x \mid n \mid e_1 \ \mathrm{op}_a \ e_2 \\
b &::= \quad \mathbf{true} \mid \mathbf{false} \mid \neg b_1 \mid b_1 \ \mathrm{op}_b \ b_2 \mid e_1 \ \mathrm{op}_c \ e_2 \\
s &::= \quad x \coloneqq e; \mid \mathbf{skip} \mid s_1 \ s_2 \mid \mathbf{if} \ b \ \mathbf{then} \ s_1 \ \mathbf{else} \ s_2 \mid \mathbf{while} \ b \ \mathbf{do} \ s_1
\end{aligned}
$$

Where:

- $n \in \mathbb{Z}$
- $\mathrm{op}_a \in \{+, \ -, \ *, \ \div\}$
- $\mathrm{op}_b \in \{\&\&, \ \|\}$
- $\mathrm{op}_c \in \{==, \ >, \ <, \ \geq, \ \leq\}$

Abstract semantics

With the sign abstract domain

- Quick recall to the collecting semantics of IMP

Abstract semantics

- We abstracted computations, shifting from a concrete domain to an abstract one

Abstract semantics

- We abstracted computations, shifting from a concrete domain to an abstract one, but we must ensure that the soundness of the abstract semantics

Soundness

- Suppose that C and A form a GC via α and γ and let $f : C \to C$ be a concrete operation and $f^\sharp : A \to A$ an abstraction. $f^\sharp$ is sound if

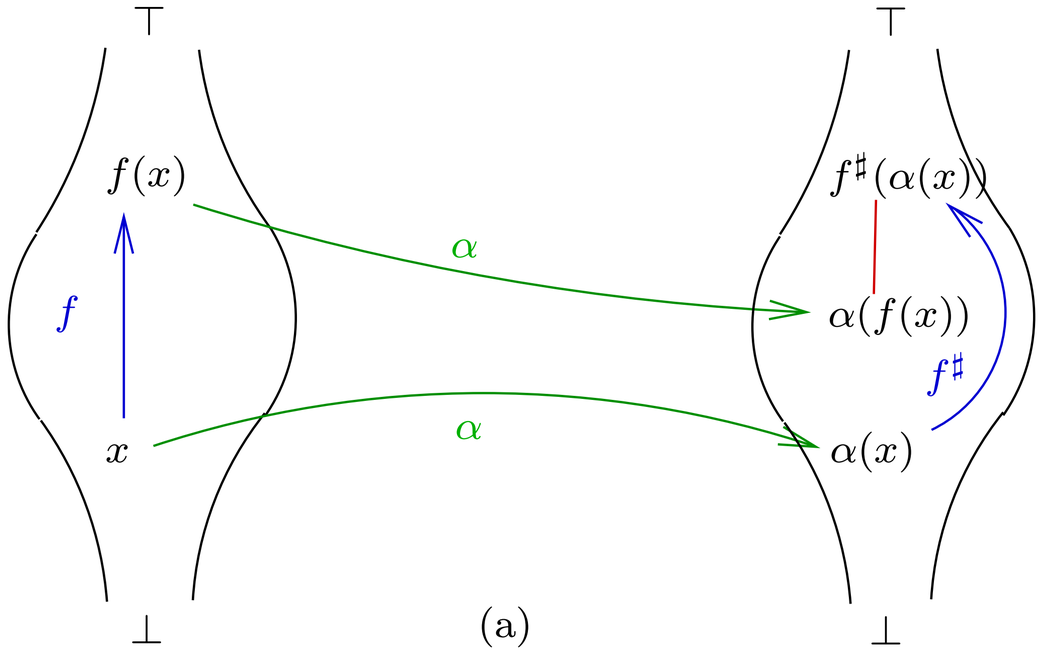
$$\forall c \in C \,.\, \alpha(f(x)) \sqsubseteq_A f^\sharp(\alpha(x))$$

or equivalently

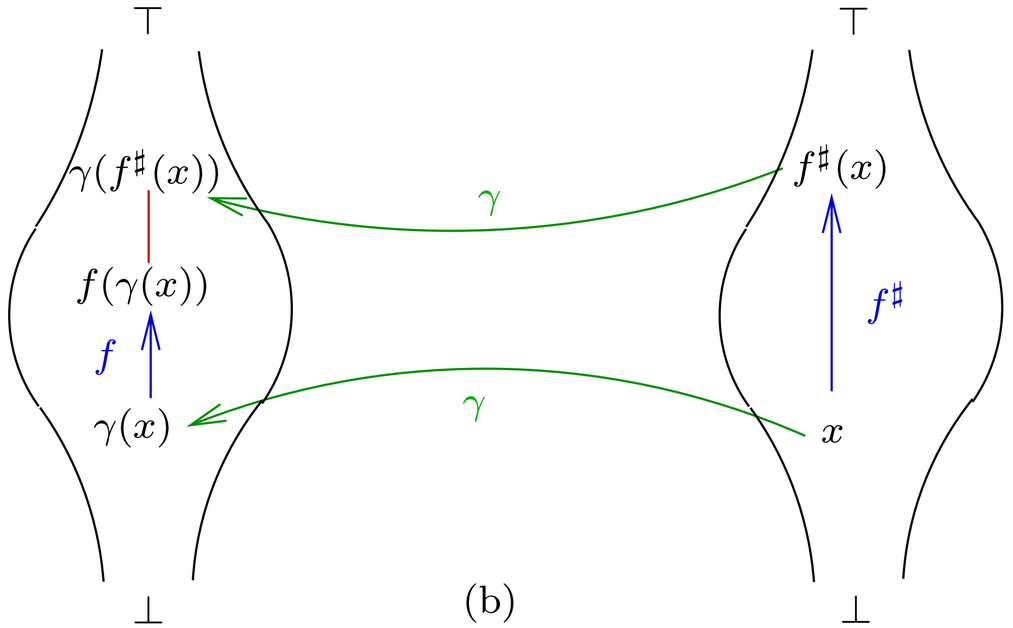
$$\forall a \in A \,.\, f(\gamma(a)) \sqsubseteq_C \gamma(f^\sharp(a))$$

Soundness

BACKWARD



FORWARD



$$\forall c \in C \,.\, \alpha(f(x)) \sqsubseteq_A f^\sharp(\alpha(x))$$

$$\forall a \in A \,.\, f(\gamma(a)) \sqsubseteq_C \gamma(f^\sharp(a))$$

Soundness

- Among all the sound $f^\sharp$ abstract functions of f, which is the *best* one?

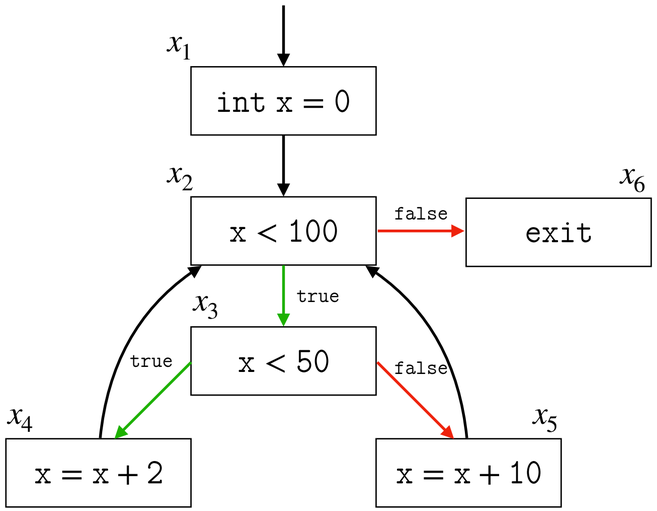
Soundness

- Among all the sound $f^\sharp$ abstract functions of f, which is the *best* one?
- Suppose that C and A form a GC via α and γ and let $f : C \to C$ be a concrete operation. The best correct approximation $f^{\text{bca}} : A \to A$ is

$$f^{\text{bca}} = \alpha \circ f \circ \gamma$$

- Unfortunately, it may not be computable
- Why?

Sign analysis



$x_1 =$

$x_2 =$

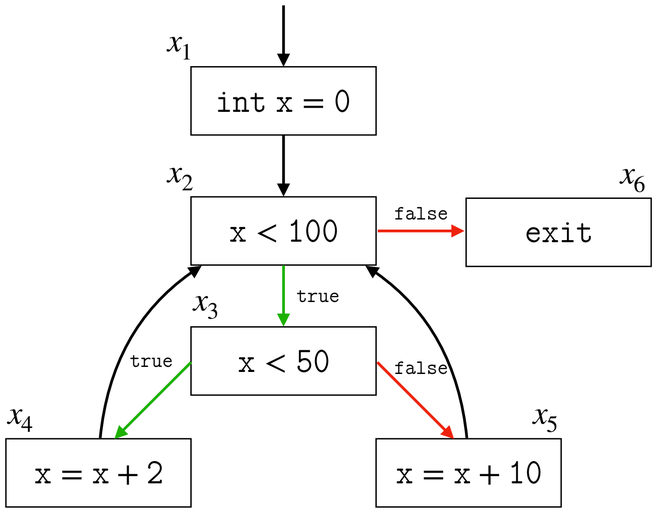
$x_3 =$

$x_4 =$

$x_5 =$

$x_6 =$

Sign analysis



$$x_1 = \alpha(I)$$

$$x_2 =$$

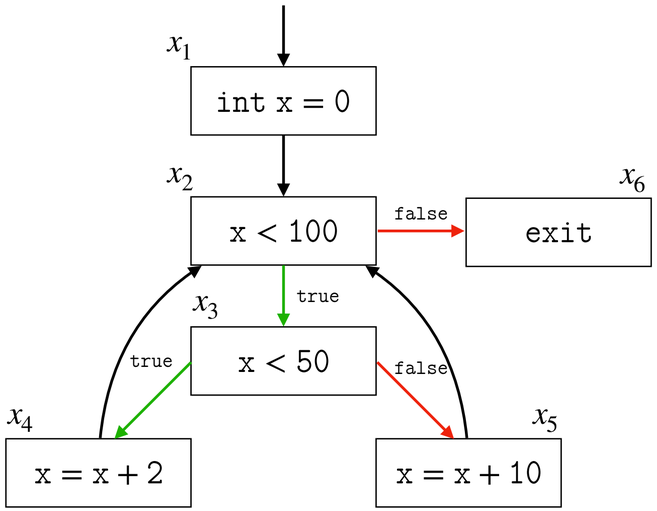
$$x_3 =$$

$$x_4 =$$

$$x_5 =$$

$$x_6 =$$

Sign analysis



$$x_1 = \alpha(I)$$

$$x_2 = [[\mathtt{x} = 0]]^\sharp x_1 \sqcup [[\mathtt{x} = \mathtt{x} + 2]]^\sharp x_4 \sqcup [[\mathtt{x} = \mathtt{x} + 10]]^\sharp x_5$$

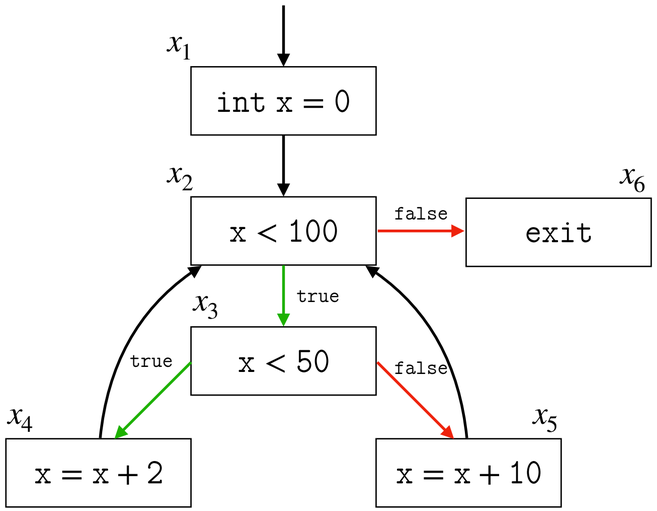
$$x_3 =$$

$$x_4 =$$

$$x_5 =$$

$$x_6 =$$

Sign analysis



$$x_1 = \alpha(I)$$

$$x_2 = [[\mathtt{x} = 0]]^\sharp x_1 \sqcup [[\mathtt{x} = \mathtt{x} + 2]]^\sharp x_4 \sqcup [[\mathtt{x} = \mathtt{x} + 10]]^\sharp x_5$$

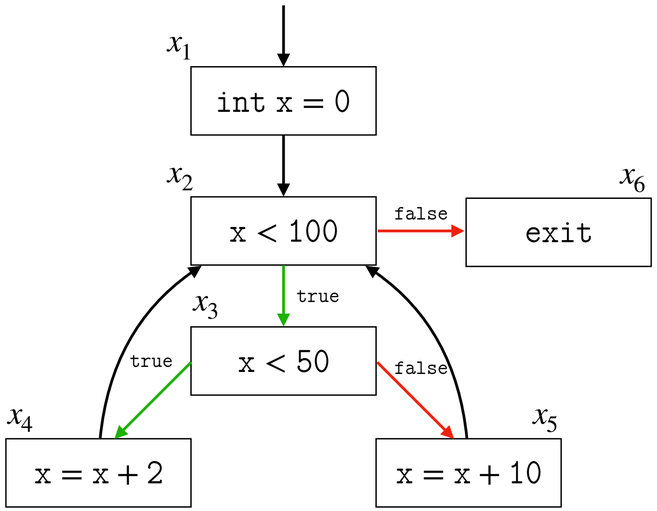
$$x_3 = [[\mathtt{x} < 100]]^\sharp x_2$$

$$x_4 =$$

$$x_5 =$$

$$x_6 =$$

Sign analysis



$$x_1 = \alpha(I)$$

$$x_2 = [[\mathtt{x} = 0]]^\sharp x_1 \sqcup [[\mathtt{x} = \mathtt{x} + 2]]^\sharp x_4 \sqcup [[\mathtt{x} = \mathtt{x} + 10]]^\sharp x_5$$

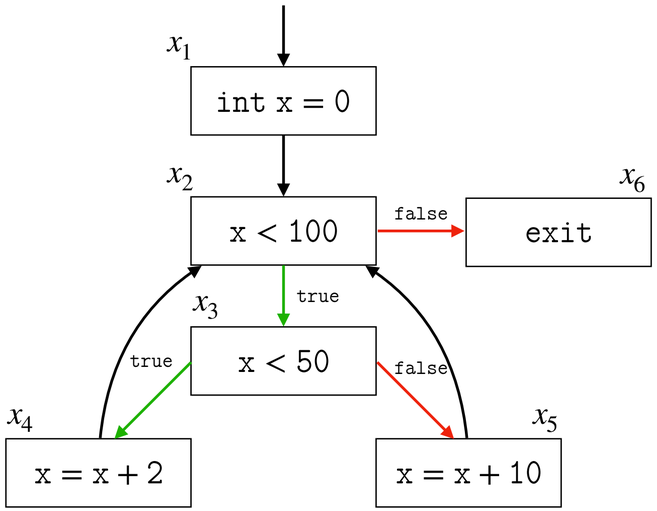
$$x_3 = [[\mathtt{x} < 100]]^\sharp x_2$$

$$x_4 = [[\mathtt{x} < 50]]^\sharp x_3$$

$$x_5 =$$

$$x_6 =$$

Sign analysis



$$x_1 = \alpha(I)$$

$$x_2 = [[\mathrm{x} = 0]]^\sharp x_1 \sqcup [[\mathrm{x} = \mathrm{x} + 2]]^\sharp x_4 \sqcup [[\mathrm{x} = \mathrm{x} + 10]]^\sharp x_5$$

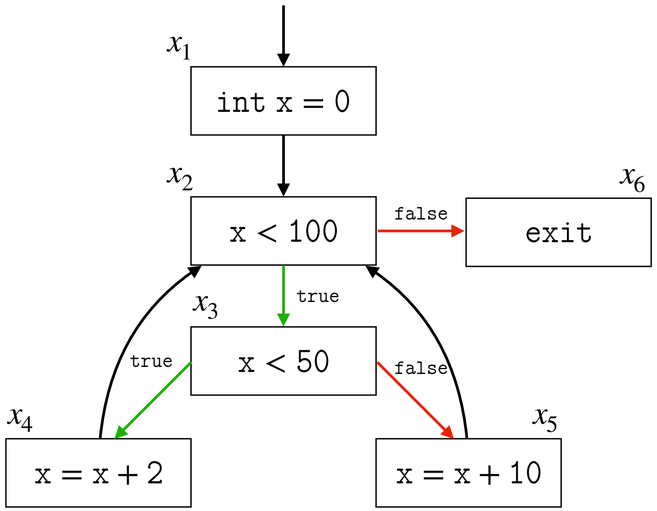
$$x_3 = [[\mathrm{x} < 100]]^\sharp x_2$$

$$x_4 = [[\mathrm{x} < 50]]^\sharp x_3$$

$$x_5 = [[\mathrm{x} >= 50]]^\sharp x_3$$

$$x_6 =$$

Sign analysis



$$x_1 = \alpha(I)$$

$$x_2 = [[\mathtt{x} = 0]]^\sharp x_1 \sqcup [[\mathtt{x} = \mathtt{x} + 2]]^\sharp x_4 \sqcup [[\mathtt{x} = \mathtt{x} + 10]]^\sharp x_5$$

$$x_3 = [[\mathtt{x} < 100]]^\sharp x_2$$

$$x_4 = [[\mathtt{x} < 50]]^\sharp x_3$$

$$x_5 = [[\mathtt{x} >= 50]]^\sharp x_3$$

$$x_6 = [[\mathtt{x} >= 100]]^\sharp x_2$$

$$
\begin{aligned}
x_1 &= \alpha(I) \\
x_2 &= [[\texttt{x} = 0]]^\sharp x_1 \sqcup [[\texttt{x} = \texttt{x} + 2]]^\sharp x_4 \sqcup [[\texttt{x} = \texttt{x} + 10]]^\sharp x_5 \\
x_3 &= [[\texttt{x} < 100]]^\sharp x_2 \\
x_4 &= [[\texttt{x} < 50]]^\sharp x_3 \\
x_5 &= [[\texttt{x} >= 50]]^\sharp x_3 \\
x_6 &= [[\texttt{x} >= 100]]^\sharp x_2
\end{aligned}
$$

- Given m nodes of a CFG, what we solve the following system of equations

$$F^\sharp = \{x_i = x_i \sqcup f_i^\sharp(x_1, \ldots, x_m) \mid i = 1,2,\ldots,m\}$$

- computing the least solution of the equation set $F^\sharp$ as the limit of a Kleene iteration

$$
\begin{aligned}
x_1 &= \alpha(I) \\
x_2 &= [[\mathtt{x} = 0]]^\sharp x_1 \sqcup [[\mathtt{x} = \mathtt{x} + 2]]^\sharp x_4 \sqcup [[\mathtt{x} = \mathtt{x} + 10]]^\sharp x_5 \\
x_3 &= [[\mathtt{x} < 100]]^\sharp x_2 \\
x_4 &= [[\mathtt{x} < 50]]^\sharp x_3 \\
x_5 &= [[\mathtt{x} >= 50]]^\sharp x_3 \\
x_6 &= [[\mathtt{x} >= 100]]^\sharp x_2
\end{aligned}
$$

- Given m nodes of a CFG, what we solve the following system of equations

$$F^\sharp = \{x_i = x_i \sqcup f_i^\sharp(x_1, \ldots, x_m) \mid i = 1,2,\ldots,m\}$$

- computing the least solution of the equation set $F^\sharp$ as the limit of a Kleene iteration over an abstract domain A

$$x^{\sharp \mathrm{lfp(F)}} = (x_1^{\sharp \mathrm{lfp}(F)}, \ldots, x_m^{\sharp \mathrm{lfp}(F)})$$

Fixpoint transfer

- Who guarantees that lfp($F^\sharp$) is a sound approximation of lfp(F)?
  - (exact) $\alpha(\text{lfp}(F)) = \text{lfp}F^\sharp$
  - (approximated) $\alpha(\text{lfp}(F)) \sqsubseteq \text{lfp}F^\sharp$

Kleenian fixpoint approximation

- If $f : C \to C$ is continuous in a complete lattice $\langle C, \sqsubseteq_C , \sqcup_C , \sqcap_C , \bot_C , \top_C \rangle$ and $f^\sharp : A \to A$ is a sound abstraction of f in a abstract domain $\langle A, \sqsubseteq_A , \sqcup_A , \sqcap_A , \bot_A , \top_A \rangle$, and the sequence $\{f_i^\sharp(\bot_A) \mid i \in \mathbb{N}\}$ has a limit $x \in A$, then it is a sound approximation of lfp(f), i.e., lfp$(f) \sqsubseteq_C \gamma(x)$
  - In particular, at each step $i, f_i^\sharp(\bot_a)$ is a sound approximation of $f_i(\bot_C)$
  - Note that the theorem guarantees soundness, not optimality

Tarskian fixpoint approximation

- Given a complete lattice $\langle C, \sqsubseteq_C , \sqcup_C , \sqcap_C , \bot_C , \top_C \rangle$, a monotonic function $f : C \to C$, and a sound abstraction $f^\sharp : A \to A$ of f in an abstract domain $\langle A, \sqsubseteq_A , \sqcup_A , \sqcap_A , \bot_A , \top_A \rangle$, then *any* post-fixpoint a of $f^\sharp$, i.e., such that $f^\sharp(a) \sqsubseteq_A a$, is a sound abstraction of lfp(f), i.e., lfp$(f) \sqsubseteq_C \gamma(a)$
  - The theorem exploits two facts
    - Tarski's characterization of lfp(f) as the smallest post-fixpoint
    - any abstract post-fixpoint of a sound abstraction represents via γ, a concrete post-fixpoint, thus an over-approximation of lfp(f)